

Artificial Intelligence for Digital Characters

Exercise 5: Autonomous Agents

Hand-out Date: 27 May 2026



Goals

- Understand the fundamentals of autonomous agents and their knowledge representation and decision-making processes.
- Understand approaches for implementing agent decision-making processes using dialogue trees and knowledge graph embeddings.
- Understand the application of reinforcement learning in non-differentiable simulation systems and robotic control tasks.
- Learn to formulate reinforcement learning problems by properly defining states, actions, rewards, and policies for complex motion tracking tasks.
- Develop practical knowledge of reinforcement learning training methodologies, including data collection strategies, policy updating algorithms, and implementation considerations.

Resources

The lecture slides, exercise slides, and Unity template are accessible via Moodle.

Tasks

1. Multiple Choice Questions

Please mark the correct answer for the multiple-choice questions below.

a) Autonomous agents that demonstrate proactiveness are primarily concerned with responding to immediate changes in their environment rather than planning for future opportunities.

☐ True ☐ False

b) Dialogue trees are highly scalable, making it easy to automatically incorporate new topics and scenarios without significant manual design work.

☐ True ☐ False

c) Link prediction in knowledge graphs is the process of determining whether a relationship should exist between two entities, which is important for applications like content recommendation and question answering.

☐ True ☐ False

d) After performing knowledge graph completion on a massive and incomplete knowledge graph, the resulting completed graph becomes dense, causing the time complexity of traversing it to increase exponentially with respect to path length.

☐ True ☐ False

e) To update the policy neural network, the transition probability, i.e., $p(s_{t+1}|s_t, a_t)$, has to be differentiable to perform back-propagation.

☐ True ☐ False

f) The Q-learning method is normally designed for discrete action spaces.

☐ True ☐ False

g) An optimal policy can maximize the reward at each step in the future.

☐ True ☐ False

h) The PPO algorithm is generally more stable than the vanilla policy gradient method.

☐ True ☐ False

2. Knowledge Graphs

You are tasked with building a knowledge graph to support a museum guide agent. Use the text below as your source.

The Renaissance is a period of history and a European cultural movement spanning the 15th and 16th centuries. It marked the transition from the Middle Ages to modernity and was characterized by efforts to revive and surpass the ideas and achievements of classical antiquity. Associated with significant social changes across many fields and disciplines, the Renaissance began in the Republic of Florence and then spread throughout Italy and later across Europe. The Renaissance's intellectual foundation was based on its version of humanism, derived from the concept of Roman humanitas and the rediscovery of classical Greek philosophy, such as that of Protagoras.

- a) Generate a knowledge graph from the text above by following these two steps:
- (i) From the list of 15 potential entities, select 10 entities that are most meaningful for the current context and explain your criteria: Renaissance, European cultural movement, 15th century, 16th century, Middle Ages, modernity, classical antiquity, ideas and achievements (of classical antiquity), Republic of Florence, Italy, Europe, humanism, Roman humanitas, classical Greek philosophy, Protagoras
 - (ii) Construct a knowledge graph using the selected entities as nodes. Define the relationships (edges) between these entities based on the text (e.g., "marked transition from", "centered in", "derived from").
- b) Using the knowledge graph you created in a), for the two questions below, first classify the type of query each question represents. Then, write SPARQL queries to retrieve the relevant information.
- (i) From which historical period did the Renaissance transition?
 - (ii) What ancient philosophical tradition formed the Renaissance's intellectual foundation?
- c) When the knowledge graph is large and incomplete, knowledge graph embeddings (KGE) can be used for information retrieval.
- (i) Assuming your KG's entities and relationships have been embedded using TransE, explain how you would find the classical Greek philosopher most similar to Protagoras in the embedding space.

- (ii) Given is the following query: "Which European cultural movements that were based on ideas derived from classical Greek philosophy AND spread across Europe after the Middle Ages?". First, identify the type of the query. Then, describe how you would use KGE to answer the query.
- d) List at least two practical situations where knowledge graphs can effectively support LLM agents.

>

3. Reinforcement Learning

Assume that you have already established a simulation platform, in which a humanoid agent and the ground plane are available. In simulation, you can control the robot to move by specifying the motor torques. When no torques are given, the robot falls to the ground due to gravity.

You are tasked to train an optimal policy with reinforcement learning, so that the humanoid can track a motion sequence of human walking, while keeping the body balance.

a) Reinforcement learning formulation.

- (i) Why should we use reinforcement learning for this task instead of supervised or unsupervised learning?
- (ii) Define the agent, the world, the action space, the reward, the state, and the policy function.

b) Describe your plan to train the policy neural network.

- (i) How can data be collected?
- (ii) How is the policy network with the Actor-critic (A2C) paradigm updated?
- (iii) Why should we terminate the simulation when the robot has fallen to the ground?
- (iv) Can we use deep Q-learning (DQN) to achieve our goal?

Project

Project Stages:

1. **Speech Recognition Integration:** Implement speech-to-text capabilities using OpenAI's Whisper to transcribe spoken words into text. Display the transcriptions either in the console or on the screen.
2. **Large Language Model (LLM) Integration:** Connect a large language model, such as Llama or DeepSeek, to enable the character to generate text-based responses.
3. **Speech Synthesis Integration:** Incorporate speech synthesis technologies like FastSpeech 2 to provide a voice for the character.
4. **Character Animation Enhancement:** Enhance the character with animations, including facial expressions using tools like Audio2Face and body animations through platforms like Mixamo.
5. **Designing and Implementing Dialog Tree:** Create a dialog tree in Twine that links user inputs to narrative paths. Implement it in Rasa by defining intents, slots, and stories to ensure smooth transitions.

Stage 5: Designing and Implementing Dialog Tree

In this stage, you will create a simple interactive storyline with branching narratives using Twine. This will serve as the foundation for your agent's dialogue system and decision-making process. Twine provides a visual editor for creating non-linear stories that can be easily translated into dialogue trees.

1. Storyline Design in Twine

- **Set up Twine**
 - Download and install Twine from <https://twinery.org/> or use the online version.
 - Launch Twine and create a new project.
- **Create Your Storyline Structure**
 - Write an introduction that establishes your agent's character and initial dialogue.
 - Design at least three different conversation paths with meaningful branches.
- **Implement Decision Points**

- For each major decision point, create multiple passages representing different responses. Ensure each branch has a logical conclusion or loops back to a central narrative point.
- Consider user intents (e.g., asking for information, expressing emotion, requesting tasks).
- Use variables if needed for more complex narratives.

- **Test Your Storyline**

- Use Twine's "Test" feature to navigate through all branches of your storyline.
- Verify that all links work correctly and lead to appropriate passages.
- Check for narrative consistency and logical flow between conversation branches.
- Refine the dialogue based on this testing.

After completing this stage, you will have a complete interactive storyline that serves as a blueprint for your agent's dialogue system.

2. Implement Chatbot in Rasa

In this stage, you will transform your Twine storyline into a functional chatbot using Rasa, an open-source machine learning framework for automated text and voice-based conversations. You will configure intents, entities, and dialogue flows based on your Twine story, then connect the Rasa chatbot to your Unity agent.

- **Set Up Rasa Environment**

- Set up a virtual environment and install [Rasa](#)
- Initialize a new Rasa project: `rasa init`

- **Define Intents and Entities**

- Analyze your Twine storyline to identify key user intents (requests, questions, responses).
- Open the `data/nlu.yml` file and define these intents with example utterances.
- Identify entities (specific information like names and locations) relevant to your story.
- Add entity examples within your intent examples using the following format: `[entity value](entity name)`.

- **Implement Dialogue Flows**

- Open `data/stories.yml` and create stories that correspond to your Twine branches.

- For each dialogue branch, define a sequence of:
 - * User intents: what the user might say
 - * Bot actions: how your agent should respond
 - * Conditions: when to take certain paths based on entities or slots
- Ensure your stories cover all the main paths from your Twine storyline.
- **Configure Response Templates**
 - Open `domain.yml` and add response templates for each bot action.
 - Structure responses to match the dialogue in your Twine story.
 - Use variations for responses to make conversations feel more natural.
 - Include `utter_actions` for each response your bot needs to give.
- **Train and Test the Model**
 - Train your Rasa model.
 - Make adjustments to NLU and dialogue flows based on testing results.
 - Iterate until the chatbot effectively implements your Twine storyline.
- **Set Up Rasa Server**
 - Run the Rasa server: `rasa run -enable-api`
 - In a separate terminal, run actions server if needed: `rasa run actions`
 - The API will be accessible at `http://localhost:5005`
- **Connect to Unity**
 - In Unity, create a script to handle HTTP requests to the Rasa API. Alternatively, you can use your existing Flask server.
 - Implement POST requests to `http://localhost:5005/webhooks/rest/webhook` with JSON payload: `{"message": "user message"}`.
 - Parse the JSON response to extract the bot's reply.
 - Play the response through your agent's dialogue system in Unity.
 - Connect the input from your Unity scene to send messages to Rasa.

After completing this stage, your agent will be able to engage in interactive conversations based on your designed storyline, processing user input, and responding according to the dialogue flows that you have defined in Rasa.

For troubleshooting, refer to the Rasa documentation at <https://rasa.com/docs/> or seek help through the course forum.